Module 3 — Class 2 Assignment: Scaling

Type-it-yourself exercise

UN AI/ML Mentorship

Module 3 — Class 2 Assignment: Scaling

Topic: Scaling — StandardScaler, MinMaxScaler, RobustScaler, leakage rule

Goal: Open a **fresh Google Colab notebook** and type the code yourself by following the steps below. Do NOT copy-paste from the working notebook. Typing it yourself is how you learn the syntax.

How to submit

1. Open Google Colab → File → New Notebook.
2. Type the code from each step below. Add a short `#` comment on EVERY cell so we know you understand it.
3. Save the notebook as `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`.
4. Push it to your team repo at `module-<X>/class_<Y>/submissions/<YourName>/`.

Step 0 — Load and pick the columns

We use the same Telco dataset but with different numeric setup.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
url = 'https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-C
df = pd.read_csv(url)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce').fillna(0)
```

Step 1 — Pick the columns and look at the problem

What to do: Show min/max of MonthlyCharges and TotalCharges — see the huge range gap.

Type this in a new cell:

```
cols = ['MonthlyCharges', 'TotalCharges']
df[cols].describe().round(2)
```

Expected output:

```
MonthlyCharges range ~18-118, TotalCharges range ~0-8684 - huge gap.
```

Step 2 — Apply MinMaxScaler (squeeze to 0-1)

What to do: Import, create the scaler, fit_transform.

Type this in a new cell:

```
from sklearn.preprocessing import MinMaxScaler
mm = MinMaxScaler()
X = df[cols]
X_mm = mm.fit_transform(X)
X_mm_df = pd.DataFrame(X_mm, columns=cols)
X_mm_df.agg(['min', 'max']).round(2)
```

Expected output:

```
Both columns: min=0.0, max=1.0
```

Step 3 — Apply StandardScaler

What to do: Should give mean ~0 and std ~1.

Type this in a new cell:

```
from sklearn.preprocessing import StandardScaler
std = StandardScaler()
X_std = std.fit_transform(X)
X_std_df = pd.DataFrame(X_std, columns=cols)
X_std_df.describe().round(2)
```

Expected output:

```
Both columns: mean ~0.0, std ~1.0
```

Step 4 — Apply RobustScaler

What to do: Uses median + IQR — better when there are outliers.

Type this in a new cell:

```
from sklearn.preprocessing import RobustScaler
rb = RobustScaler()
X_rb = rb.fit_transform(X)
X_rb_df = pd.DataFrame(X_rb, columns=cols)
X_rb_df.describe().round(2)
```

Expected output:

```
Median is 0 for both columns (robust scaling centers on median).
```

Step 5 — Compare distributions with histograms

What to do: Plot the original vs scaled distributions side by side.

Type this in a new cell:

```
fig, ax = plt.subplots(1, 4, figsize=(16, 4))
ax[0].hist(df['MonthlyCharges'], bins=30); ax[0].set_title('Original')
ax[1].hist(X_mm_df['MonthlyCharges'], bins=30); ax[1].set_title('MinMax')
ax[2].hist(X_std_df['MonthlyCharges'], bins=30); ax[2].set_title('Standard')
ax[3].hist(X_rb_df['MonthlyCharges'], bins=30); ax[3].set_title('Robust')
plt.tight_layout()
plt.show()
```

Expected output:

```
4 side-by-side histograms - same SHAPE but different x-axis scales.
```

Step 6 — The leakage rule: split FIRST, scale SECOND

What to do: Split, then fit_transform on TRAIN, transform on TEST.

Type this in a new cell:

```
from sklearn.model_selection import train_test_split
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
print('Train mean:', X_train_s.mean(axis=0).round(2))
print('Test mean:', X_test_s.mean(axis=0).round(2))
```

Expected output:

```
Train mean ~0.0, test mean close to 0 but not exactly.
```

Checklist before you submit

- ☐ All cells run without error
- ☐ Every code cell has a `#` comment in your own words
- ☐ Outputs are visible (you saved AFTER running)
- ☐ File name is correct: `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`

Deadline: before next class.